ACM DIGITAL LIBRARY
Association for Computing Machinery
acm open

Latest updates: https://dl.acm.org/doi/10.1145/3715784

PDF Download
3715784.pdf
13 January 2026
Total Citations: 1
Total Downloads: 682

Published: 19 June 2025
Accepted: 14 January 2025
Received: 13 September 2024

Citation in BibTeX format

RESEARCH-ARTICLE

Detecting and Reducing the Factual Hallucinations of Large Language Models with Metamorphic Testing

WEIBIN WU, Sun Yat-Sen University, Guangzhou, Guangdong, China
YUHANG CAO, Sun Yat-Sen University, Guangzhou, Guangdong, China
NING YI, Sun Yat-Sen University, Guangzhou, Guangdong, China
RONGYI OU, Sun Yat-Sen University, Guangzhou, Guangdong, China
ZIBIN ZHENG, Sun Yat-Sen University, Guangzhou, Guangdong, China

Open Access Support provided by:
Sun Yat-Sen University

Detecting and Reducing the Factual Hallucinations of Large Language Models with Metamorphic Testing

WEIBIN WU, Sun Yat-sen University, China
YUHANG CAO, Sun Yat-sen University, China
NING YI, Sun Yat-sen University, China
RONGYI OU, Sun Yat-sen University, China
ZIBIN ZHENG*, Sun Yat-sen University, China

Question answering (QA) is a fundamental task of large language models (LLMs), which requires LLMs to automatically answer human-posed questions in natural language. However, LLMs are known to distort facts and make non-factual statements (i.e., hallucinations) when dealing with QA tasks, which may affect the deployment of LLMs in real-life situations. In this work, we propose DrHall, a framework for detecting and reducing the factual hallucinations of black-box LLMs with metamorphic testing (MT). We believe that hallucinated answers are unstable. Therefore, when LLMs hallucinate, they are more likely to produce different answers if we use metamorphic testing to make LLMs re-execute the same task with different execution paths, which motivates the design of DrHall. The effectiveness of DrHall is evaluated empirically on three datasets, including a self-built dataset of natural language questions: FactHalluQA, as well as two datasets of programming questions: Refactory and LeetCode. The evaluation results confirm that DrHall can consistently outperform the state-of-the-art baselines, obtaining an average F1 score of over 0.856 for hallucination detection. For hallucination correction, DrHall can also outperform the state-of-the-art baselines, with an average hallucination correction rate of over 53%. We hope that our work can enhance the reliability of LLMs and provide new insights for the research of LLM hallucination mitigation.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Large Language Model, Hallucination, Metamorphic Testing

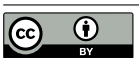
ACM Reference Format:
Weibin Wu, Yuhang Cao, Ning Yi, Rongyi Ou, and Zibin Zheng. 2025. Detecting and Reducing the Factual Hallucinations of Large Language Models with Metamorphic Testing. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE065 (July 2025), 22 pages. https://doi.org/10.1145/3715784

1 Introduction

Thanks to the rapid advancement of natural language processing (NLP) techniques, deep neural network-based virtual assistants like Apple's Siri [Apple Inc. 2023] and Baidu's Xiaodu [Baidu Inc. 2023] have become increasingly popular in recent years, facilitating more natural human-machine interactions. Question answering (QA) tasks play a crucial role in these interactions [Clark et al. 2019; Hirschman and Gaizauskas 2001; Xiong et al. 2016]. It is thus essential for machines to

*Corresponding author.

Authors' Contact Information: Weibin Wu, Sun Yat-sen University, Zhuhai, China, wuwb36@mail.sysu.edu.cn; Yuhang Cao, Sun Yat-sen University, Zhuhai, China, caoyh33@mail2.sysu.edu.cn; Ning Yi, Sun Yat-sen University, Zhuhai, China, yining@mail2.sysu.edu.cn; Rongyi Ou, Sun Yat-sen University, Zhuhai, China, oury@mail2.sysu.edu.cn; Zibin Zheng, Sun Yat-sen University, Zhuhai, China, zhzibin@mail.sysu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2994-970X/2025/7-ARTFSE065
https://doi.org/10.1145/3715784

comprehend natural language and provide responses to queries posed by humans in a natural manner that is similar to humans [Botana et al. 2023].

The emergence of large language models (LLMs) has presented novel possibilities for the advancement of virtual assistants [Gondala et al. 2021; Piñeiro-Martín et al. 2023]. Nevertheless, the issue of hallucinations incurred by LLMs is alarming [Amodei et al. 2016; Hendrycks et al. 2021]. Specifically, LLMs may offer erroneous responses to a given query, which can compromise user satisfaction and even cause legal issues [Devaraj et al. 2022; Kaye 2023; Tam et al. 2022].

Therefore, it is essential to detect hallucinations of LLMs in QA tasks in a timely and automatic manner. Hallucinations are typically classified into two main categories: factuality hallucinations and faithfulness hallucinations. Factuality hallucinations occur when there are differences between the generated content and verifiable real-world facts. By contrast, faithfulness hallucinations occur when there are differences between the generated content and the input contexts [Huang et al. 2025]. Most previous research mainly focuses on identifying faithfulness hallucinations [Huang et al. 2023; Miao et al. 2023; Shen et al. 2022]. Nevertheless, in practice, a considerable amount of QA tasks do not include contexts, making factuality hallucinations more frequently occur than faithfulness hallucinations [Chen and Shu 2024; Wang et al. 2023, 2024; Zhao et al. 2023].

Existing methods for factuality hallucination detection only achieve limited success. Numerous works utilize the model's confidence score to detect factuality hallucinations [Luo et al. 2023; Varshney et al. 2023]. For black-box LLMs that do not provide the confidence scores [OpenAI Inc. 2023], some works propose to infer LLMs' confidence scores based on their observable behaviors [Kadavath et al. 2022; Manakul et al. 2023; Xiong et al. 2024]. For example, SelfCheckGPT obtains various LLM responses and compares their consistency to infer the LLMs' confidence scores [Manakul et al. 2023]. However, the reliability of LLMs' confidence scores is related to their performance [Orgad et al. 2024; Zhang et al. 2023]. Therefore, when LLMs hallucinate, their confidence scores can also be unreliable, making it inappropriate to detect hallucinations based on confidence scores [Huang et al. 2025].

In this work, we focus on the factuality hallucinations of LLMs when dealing with QA tasks and propose DrHall, a framework for **D**etecting and **r**educing the factual **Hall**ucinations of black-box LLMs with metamorphic testing (MT) [Chen et al. 2020, 2018]. We believe that hallucinated answers are unstable [Huang et al. 2025; Sclar et al. 2023]. Therefore, when LLMs hallucinate, they are more likely to produce different answers if we use metamorphic testing to make LLMs re-execute the same task with different execution paths. In order to induce different execution paths, we carefully design metamorphic relations (MRs) that take into account the mechanisms behind the question answering capability of LLMs, including question comprehension, knowledge recall, and knowledge reasoning [Tan et al. 2023; Wei et al. 2022; Zheng et al. 2023]. We finally obtain six base and three composite MRs. Table 1 demonstrates our MRs. By investigating the consistency between the original response and the follow-up responses under metamorphic testing, we can detect the factual hallucinations of black-box LLMs. We further propose to correct the factual hallucinations of black-box LLMs by combining our MRs with multi-path voting.

We evaluate the effectiveness of DrHall with three QA datasets. For natural language QA tasks, to avoid the impact of benchmark leakage on the evaluation results and characterize more realistic application scenarios [Zhou et al. 2023b], we construct a new dataset, FactHalluQA, using knowledge from the Wikipedia. Besides, we consider programming QA tasks and evaluate DrHall's ability to deal with code hallucinations, which refer to the phenomenon where the code generated by LLMs is syntactically correct or even semantically plausible but actually does not work as desired [Tian et al. 2024]. Specifically, we focus on two categories of programming questions: program repair and code generation. Figure 1 illustrates a sample programming question and the code hallucination

Table 1. Demonstrations of the hallucination detection process for GPT-3.5-turbo with base MRs. The Original Question is the original inquiry posed by the user to the LLM. The Golden Answer is the ground-truth response provided by the dataset. The Original Answer is the answer given by the LLM in response to the original question, which requires further testing to ascertain whether it is a hallucination. The Follow-up Question and the Follow-up Answer represent the question generated with our MRs and the answer provided by the LLM in response to the follow-up question, respectively.

Original Question: Who is the famous Korean Go player with the nickname "Stone Buddha"? **Golden Answer**: Lee Chang-ho **Original Answer (Processed)**: Cho Hun-hyun
QMR1 Example
Follow-up Question: Who is the famous ... Please think step by step to solve this problem and give the answer. **Follow-up Answer**: The first step is to identify the famous Korean Go player with the nickname "Stone Buddha." One of the most well-known Korean Go players who is often referred to as "Stone Buddha" is Lee Changho. Therefore, the famous Korean Go player with the nickname "Stone Buddha" is Lee Changho. **Follow-up Answer (Processed)**: Lee Chang-ho
QMR2 Example
Follow-up Question: (es) ¿Quién es el famoso jugador de Go coreano apodado "Buda de Piedra"? (de) Wer ist der berühmte koreanische Go-Spieler mit dem Spitznamen "Stone Buddha"? (nl) Wie is de beroemde Koreaanse Go-speler met de bijnaam "Stone Buddha"? **Follow-up Answer**: (es) Der berühmte koreanische Go-Spieler mit dem Spitznamen "Stone Buddha" ist Lee Chang-ho. (de) El famoso jugador de Go coreano apodado "Buda de Piedra" es Lee Chang-ho. (nl) De beroemde Koreaanse Go-speler met de bijnaam "Stone Buddha" is Cho Hun-hyun. **Follow-up Answer (Processed)**: Lee Changho
QMR3 Example
Follow-up Question: Which Korean Go player is known as the "Stone Buddha"? **Follow-up Answer**: Choi Chul-han is known as the "Stone Buddha" in the world of Go. **Follow-up Answer (Processed)**: Choi Chul-han
QMR4 Example
Follow-up Question: Known: [Cho Chihun in Korean, is among the best players of the 20th century.] May I ask: [Who is the famous ...] **Follow-up Answer**: The famous Korean Go player with the nickname "Stone Buddha" is Cho Chihun. **Follow-up Answer (Processed)**: Cho Chihun
AMR1 Example
Follow-up Question: [The famous Korean Go player with the nickname "Stone Buddha" is Cho Hun-hyun.] Is this statement not true? **Follow-up Answer**: No, the statement is not true. Lee Chang-ho, a renowned South Korean Go player, is known by the nickname "Stone Buddha." **Follow-up Answer (Processed)**: No, the statement is not true.
AMR2 Example
Follow-up Question: Who is the famous ... Is it "Park Jeong-hwan"? Or "Lee Se-dol"? Or "Cho Hun-hyun"? Or maybe none of the above answers are correct? **Follow-up Answer**: The famous Korean Go player with the nickname "Stone Buddha" is Park Jeong-hwan. **Follow-up Answer (Processed)**: Park Jeong-hwan

generated by LLMs. Experimental results show that DrHall can consistently outperform the state-of-the-art baselines, obtaining an average F1 score of over 0.856 for hallucination detection. For hallucination correction, DrHall can also outperform the state-of-the-art baselines, with an average hallucination correction rate of over 53%.

Our contributions can be summarized as follows:

- We propose DrHall, a framework for detecting and reducing the factual hallucinations of black-box LLMs with metamorphic testing. We carefully design six base and three composite MRs for DrHall. Based on these MRs, DrHall can detect and correct the factual hallucinations of black-box LLMs by consistency assessment and multi-path voting, respectively.
- We build a new natural language QA dataset, FactHalluQA, to avoid the impact of benchmark leakage on the evaluation results. FactHalluQA consists of more than eight hundred questions in English, covering seven different subjects.
- We conducted extensive experiments to validate our framework's effectiveness. In addition to natural language QA tasks, we also consider programming QA tasks, which have not been explored by existing factual hallucination mitigation methods. Experimental results show that our framework can outperform the state-of-the-art baselines and achieve outstanding hallucination detection and correction performance.

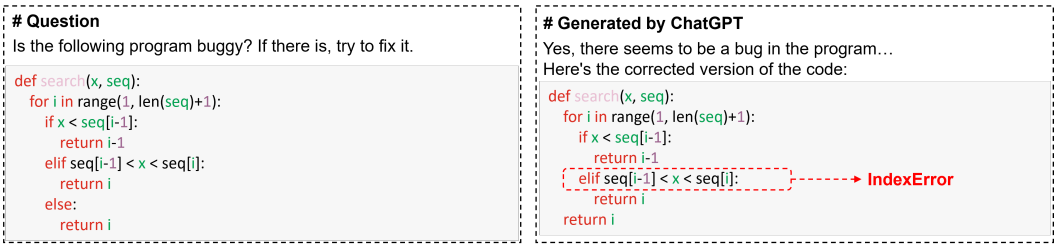


Fig. 1. A sample programming question and the code hallucination generated by LLMs.

The remainder of this paper is organized as follows: Section 2 introduces the background of this paper and the related work. Section 3 describes our technical details. Section 4 presents the experimental setup. Section 5 reports and analyzes the experimental results. Section 6 concludes this paper.

2 Background and Related Work

Detection of Factuality Hallucinations. Despite their impressive performance, deep learning models are known to have reliability [Deng et al. 2023; Wu et al. 2024] and privacy issues [Huang et al. 2024; Wu et al. 2023]. One distinct reliability issue of LLMs is that they sometimes hallucinate. Therefore, detecting hallucinations of LLMs is crucial to ensure the reliability and credibility of the generated content. To effectively detect factuality hallucinations of LLMs, an intuitive strategy is to compare model-generated content with reliable knowledge sources [Chen et al. 2024; Chern et al. 2023; Guo et al. 2022; Min et al. 2023]. For example, Chern et al. [Chern et al. 2023] propose a unified framework that allows LLMs to detect factual errors by collecting evidence using a set of external tools. However, this approach requires access to external databases and can incur significant inference costs. A current common approach is uncertainty estimation, which argues that LLM hallucinations arise from model uncertainty [Luo et al. 2023; Varshney et al. 2023]. Varshney et al. [Varshney et al. 2023] determine the model's uncertainty about key concepts by considering the minimum token probability in these concepts. However, token-level probability distributions over outputs are often unavailable when the LLM is accessible only through API calls. Because of this limitation, some researchers propose to infer the confidence score of a model from its observable behaviors [Kadavath et al. 2022; Manakul et al. 2023; Xiong et al. 2024]. For example, Manakul et al. [Manakul et al. 2023] detect hallucinations by sampling multiple responses of LLMs for the same prompt and evaluating the consistency between factual statements.

Metamorphic Testing. Metamorphic testing (MT) is a black-box testing methodology. It first establishes metamorphic relations (MRs) based on the domain knowledge of the software under test. Then it utilizes the metamorphic relations to generate new test cases and determines whether the test passes by verifying whether the metamorphic relation is maintained. Metamorphic testing is widely used to test deep learning (DL) models for different tasks due to its ability to alleviate the test oracle problem during software testing [Liu et al. 2014]. Automated driving and neural machine translation models are two typical DL models that attract many MT-based testing approaches [Luu et al. 2022; Raunak et al. 2022; Wang and Su 2020]. MT has also been applied to the question answering models. However, existing methods either do not have enough test accuracy or focus on verifying the faithfulness of the question answering model [Chen et al. 2021; Shen et al. 2022].

Chain of Thought and Self-Consistency. Chain of thought (CoT) is a technique that introduces intermediate reasoning steps to achieve complex reasoning ability [Wei et al. 2022]. Research has demonstrated that this technique can substantially enhance the performance of models on math,

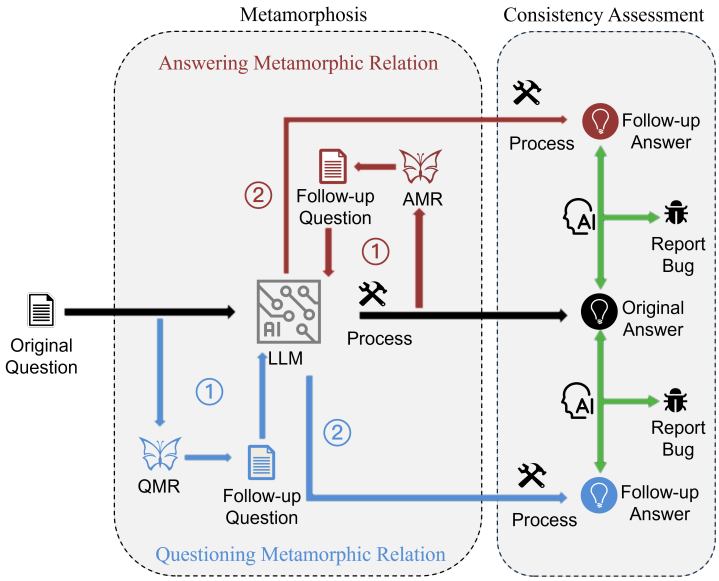


Fig. 2. A high-level overview of DrHall.

common sense, and reasoning. The reason is that CoT enables the model to decompose problems into more manageable sub-problems, which improves problem understanding and solving abilities. Numerous studies focus on enhancing the performance of the LLM using the concept of CoT [Kojima et al. 2022; Liu et al. 2022; Wang et al. 2022]. One of the most advanced methodologies is Self-Consistency [Wang et al. 2022], which asks the model to take different reasoning paths and then select the most consistent answer.

3 Approach

3.1 Overview

The core idea of DrHall is based on metamorphic testing, which does not require manual annotation of factual labels for questions. Instead, DrHall tests LLMs by asking follow-up questions and checking whether the expected metamorphic relationship is satisfied between the original and follow-up answers. As such, testing LLMs can be done in real-time.

We believe that hallucinated answers are unstable. Therefore, when LLMs hallucinate, they are more likely to produce different answers if we use metamorphic testing to make LLMs re-execute the same task with different execution paths. Different from traditional software, the execution path of LLMs is the series of internal decisions and computational steps taken by the LLM when processing a given input. The execution path is internal to the model and usually invisible to the user, but it determines how the model responds to a given input. For a black-box LLM, we can only input questions and obtain the corresponding answers from the LLM. Therefore, our designed metamorphic relations mainly work on the original question or the original answer. The formula of our metamorphic testing process is defined as follows:

$$a = P(q), \tag{1}$$

$$a' = P(QMR(q)), \tag{2}$$

$$a' = P(AMR(a)), \tag{3}$$

where q denotes the original question, and a denotes the original answer produced by the LLM P in response to the original question. QMR and AMR signify the Questioning Metamorphic Relation (QMR) and Answering Metamorphic Relation (AMR) in our framework, respectively. a' is the follow-up answer generated by the LLM in response to the follow-up question constructed with the QMR or AMR. If a and a' do not satisfy the corresponding metamorphic relation, we detect a hallucination of the LLM.

Figure 2 presents the overview of DrHall, which includes two primary phases. The metamorphosis phase uses QMRs and AMRs to obtain follow-up questions and follow-up answers. The answer consistency assessment phase verifies whether the follow-up answers meet the corresponding metamorphic relationship with the original answers. We first present the designed base MRs for original questions and original answers in Section 3.2 and Section 3.3, respectively. Then, we introduce how to obtain composite MRs in Section 3.4. After that, we present the methodology for assessing answer consistency in Section 3.5. Lastly, we introduce how to extend DrHall to correct detected factual hallucinations in Section 3.6.

3.2 Questioning Metamorphic Relation

We devise four MRs for the original questions, i.e., the Questioning Metamorphic Relations (QMRs). The goal of QMRs is to make LLMs re-execute the same task with different execution paths. To this end, we consider the mechanisms behind the question answering capability of LLMs, including question comprehension, knowledge recall, and knowledge reasoning [Tan et al. 2023; Wei et al. 2022; Zheng et al. 2023]. Question comprehension refers to a model's ability to understand the context and intent of a given question. Knowledge recall is the ability of a model to retrieve the necessary basic knowledge from memory to solve a question. Knowledge reasoning is the ability of a model to use recalled or supplied knowledge to derive new information that can solve a question. We detail our design of QMRs as follows.

QMR1: Chain of Thought. Chain of Thought (CoT) is a technique that employs intermediate reasoning steps to attain complex reasoning capabilities. We use it to enhance the model's reasoning capability in QA tasks. By inducing model reasoning, we can alter the model's execution path. Based on the zero-sample CoT, the follow-up question is constructed by adding the instruction ("Please think step by step to solve this problem and give the answer.") to the original question. Since the follow-up question is semantically equivalent to the original question, the follow-up and original answers should also be consistent. Otherwise, a hallucination is detected.

Table 1 (QMR1 Example) uses an example to illustrate the hallucination detection process with QMR1. In this example, the original question is: "Who is the famous Korean Go player with the nickname 'Stone Buddha'?" The simplified original and follow-up answers given by GPT-3.5-turbo are "Cho Hun-hyun" and "Lee Chang-ho", respectively. Therefore, an inconsistency occurs, showing that GPT-3.5-turbo's answer to the original question is not credible.

QMR2: Multilingual Voting. LLMs often use multilingual data for training [Brown et al. 2020; Ouyang et al. 2022]. Therefore, the same question posed in different languages is likely to reference different knowledge bases and result in distinct execution paths. We refer to the correct rates of LLMs' answers in each language in the performance evaluation paper [Tan et al. 2023] and choose three languages with higher correct rates, i.e., Spanish, German, and Dutch. We then generate three follow-up questions by translating the original question into the three languages via Google Translate, respectively. The outputs obtained from the three follow-up questions are translated back into English for voting, and the answer with the highest voting score is used as the follow-up answer of QMR2.

QMR3: Problem Optimization. Using LLM's powerful language processing capabilities, we alter the words and structure of the original question to achieve problem optimization. The resulting follow-up questions meet the standards of improved clarity while preserving the semantics of the original question. Through QMR3, we paraphrase the original question. This affects the question comprehension ability of the large model, ultimately changing the model's execution path. Since we require that the optimized problem maintains the semantics of the original problem, the follow-up and original answers should also be consistent. Otherwise, a hallucination is detected.

Table 1 (QMR3 Example) uses an example to illustrate the hallucination detection process with QMR3. We ask GPT-3.5-turbo to optimize the original problem with the following prompt: "I hope you can serve as a copywriter and text polishing officer. I will send you the English text, and you can help me improve the version. I hope you can change the words and structure of the sentence as much as possible. Keep the same meaning, but make them easier to understand. You only need to polish the content without explaining the questions and requirements raised in the content." After optimization, the original question becomes: "Which Korean Go player is known as the 'Stone Buddha'?" The simplified original and follow-up answers given by GPT-3.5-turbo are "Cho Hun-hyun" and "Choi Chul-han", respectively. Therefore, an inconsistency occurs, showing that GPT-3.5-turbo's answer to the original question is not credible.

QMR4: Adding External Knowledge. This MR concentrates on inducing different execution paths by altering the model's ability to recall knowledge. For natural language questions, we search Wikipedia for entities in the original question and use the retrieved sentence as evidence. Then the follow-up question can be constructed by the following form: "Known: [Evidence] May I ask: [Original Question]." Since entities do not exist in code, the entities used in dealing with programming questions are given by the model with the prompt: "Please summarize the methods that may be used in the above problem, and give the name of the methods."

With the above design, we can more efficiently identify the evidence needed to solve the original problem. The evidence not only supplements the model's knowledge, but also helps the model to more precisely recall the knowledge needed to solve the problem in its knowledge base. This MR does not change the original meaning of the original question, so the follow-up and original answers should also be consistent. Otherwise, a hallucination is detected.

3.3 Answering Metamorphic Relation

We design two MRs for the original answers, i.e., the Answering Metamorphic Relations (AMRs). Similar to QMRs, AMRs also attempt to trigger different execution paths of LLMs. We detail our design of AMRs as follows.

AMR1: Construction of General Questions. In this MR, we question the original answer to form a general question as the follow-up question. Specifically, the follow-up question is constructed by: "[Original Answer] Is this statement not true?" Since the answers given by LLM are more inclined to pander to the user [Sharma et al. 2023], choosing the negative form of questioning is more likely to make the model rethink the original answer. In this MR, the follow-up answer should support the original answer. Otherwise, a hallucination is detected.

AMR2: Construction of Multi-Choice Questions. This MR first obtains multiple distractors based on the original answer and then constructs a multi-choice question as the follow-up question with both the obtained distractors and the original answer as options. We obtain multiple distractors with the help of LLMs' powerful language processing capabilities and large knowledge base. Specifically, we first ask LLMs to simplify the original answer into the form of words or phrases. We then ask LLMs to generate multiple distractors based on the simplified answer. Note that this MR is only suitable for natural language questions, since programming questions do not have appropriate distractors.

Table 2. Demonstrations of the hallucination detection process for GPT-3.5-turbo with composite MRs. The red portion indicates the Chain of Thought section of the CMR. The **bold** portion indicates the Problem Optimization section of the CMR. The blue portion indicates the Multilingual Voting section of the CMR. The *italic* portion indicates the Adding External Knowledge section of the CMR. The original and follow-up answers are processed for simplicity.

Original Question: Who is the famous Korean Go player with the nickname "Stone Buddha"? **Golden Answer**: Lee Chang-ho **Original Answer**: Cho Hun-hyun
CMR1 Example
Follow-up Question: **Which Korean Go player is known as the "Stone Buddha"?** Please think step by step to solve this problem and give the answer. **Follow-up Answer**: Lee Chang-ho
CMR2 Example
Follow-up Question: (es) **¿Qué jugador de Go coreano es conocido ...** Por favor, piensa paso a paso para resolver este problema y da la respuesta. (de) **Welcher koreanische Go-Spieler ...** Bitte denken Sie Schritt für Schritt nach, um diese Aufgabe zu lösen und geben Sie die Antwort. (nl) **Welke Koreaanse Go-speler ...** Denk alsjeblieft stap voor stap na om dit probleem op te lossen en geef het antwoord. **Follow-up Answer**: Lee Changho
CMR3 Example
Follow-up Question: (es) *Conocido: [Cho Chihun en coreano ...]* **¿Puedo preguntar: [¿Qué jugador de Go coreano es conocido ...** Por favor ...] (de) *Bekannt: [Er ist als "Dol-bucheo" (Stein-Buddha) bekannt ...]* **Darf ich fragen: [Welcher koreanische Go-Spieler ist als "Stein-Buddha" ...** Bitte denken ...] (nl) *Bekannt: [Dieser Artikel gibt einen Überblick über bekannte ...]* **Darf ich fragen: [Welcher koreanische Go-Spieler ist als "Stein-Buddha" ...** Denk alsjeblieft ...] **Follow-up Answer**: Lee Changho

Table 1 (Example AMR2) uses an example to illustrate the hallucination detection process with AMR2. We use the prompt ("Try to summarize the answer into one word or one phrase.") to ask GPT-3.5-turbo to simplify the original answer and then ask it to generate multiple distractors with the prompt ("Please provide two phrases similar to [...]. Note: The phrases provided should be of a similar type to the original phrase, but the meaning cannot be identical. I want you to only reply with similar phrases, without writing any explanations."). We finally obtain the follow-up question with both the distractors and the original answer as the options. We can see that GPT-3.5-turbo's answer to the follow-up question is different from the original answer, indicating that GPT-3.5-turbo's response to the original question is not reliable.

3.4 Composite Metamorphic Relation

Single base MRs may fail to trigger different execution paths. Therefore, we propose to combine base MRs to obtain the Composite Metamorphic Relations (CMRs). Specifically, we stack QMRs one by one in the order of Chain of Thought (QMR1), Problem Optimization (QMR3), Multilingual Voting (QMR2), and Adding External Knowledge (QMR4), which results in three CMRs: CMR1 (QMR1 + QMR3), CMR2 (QMR1 + QMR2 + QMR3), and CMR3 (QMR1 + QMR2 + QMR3 + QMR4), respectively. Table 2 demonstrates the hallucination detection process for GPT-3.5-turbo with CMRs.

3.5 Answer Consistency Assessment

Answer consistency assessment is designed to check whether the follow-up answer satisfies the expected metamorphic relation with the original answer. In this paper, we propose an automatic answer consistency assessment method with the assistance of GPT-3.5-turbo.

First, we process the model output. LLMs' answers are often long, and irrelevant statements may interfere with the judgment of answer consistency. To address this issue, we use GPT-3.5-turbo to simplify the answers given by the model with the prompt ("Please try to answer in one sentence."). The simplified original and follow-up answers are used to assess consistency. For programming questions, it is difficult to judge the consistency between the code given by the model. We deal with this issue by giving the code the same input that conforms to the input format with the help

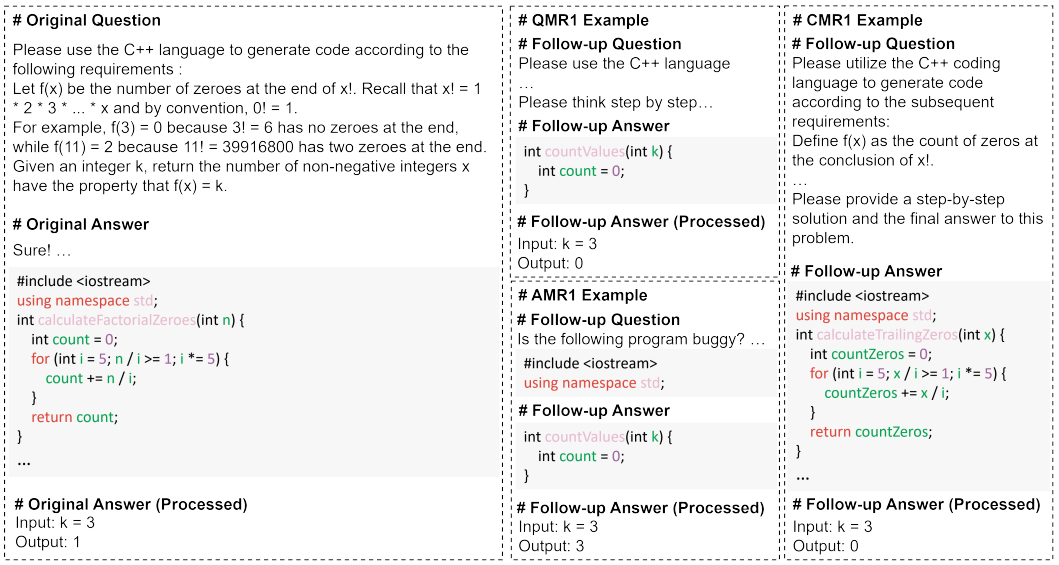


Fig. 3. Demonstrations of the hallucination detection process for GPT-3.5-turbo when dealing with programming questions with our MRs.

of a programming tool and running the code to obtain the output as the answer for consistency assessment. Figure 3 demonstrates the hallucination detection process for programming QA tasks.

We then ask GPT-3.5-turbo to assess the consistency between the processed original and follow-up answers with the following prompt: "Are the above two answers to the question consistent? Please answer in as few words or phrases as possible. Do not evaluate the rightness or wrongness of the answer itself. Note: Yes and right have the same meaning; No and Wrong have the same meaning."

3.6 Hallucination Correction

After hallucinations are detected, it is desirable for DrHall to possess the capability to correct them. We propose to extend DrHall to correct detected factual hallucinations with multi-path voting. Specifically, we first adapt the proposed MRs above to design five metamorphic relations for error correction, i.e., the Error Correction Metamorphic Relations (ECMRs). We then trigger multiple execution paths using these ECMRs and select the most frequent answer among the generated results as the corrected answer, which achieves the goal of error correction. Table 3 and Figure 4 show the examples of our designed ECMRs. We detail our ECMRs as follows:

ECMR1: Multi-Path AMR1. ECMR1 implements a multi-path version of AMR1. The original execution path of AMR1 is triggered by negatively questioning the original answer. In addition to the original execution path of AMR1, we introduce two additional paths by positively questioning and neutrally questioning.

ECMR2: Multi-Path QMR2. ECMR2 implements a multi-path version of QMR2. Since QMR2 itself has triggered multiple execution paths by translating the original question into three languages, ECMR2 adds one more language (French) for question translations to supplement the original execution paths of QMR2. Therefore, ECMR2 induces the model to respond with different execution paths corresponding to different linguistic data.

Table 3. Demonstrations of the hallucination correction process for GPT-3.5-turbo with ECMRs. The original and follow-up answers are processed for simplicity.

Original Question: Who is the famous Korean Go player with the nickname "Stone Buddha"? **Golden Answer**: Lee Chang-ho	**Original Answer**: Cho Hun-hyun
ECMR1 Example	
Follow-up Question 1: [The famous Korean Go player ...] Is this statement not true? **Follow-up Question 2**: [The famous Korean Go player ...] Is this statement true? **Follow-up Question 3**: [The famous Korean Go player ...] Please analyze this sentence. **Corrected Answer**: Not True	**Follow-up Answer 1**: Not True **Follow-up Answer 2**: Not True **Follow-up Answer 3**: True
ECMR2 Example	
Follow-up Question 1: (es) ¿Quién es el famoso jugador de Go coreano apodado "Buda de Piedra"? **Follow-up Question 2**: (de) Wer ist der berühmte koreanische Go-Spieler mit dem Spitznamen "Stone Buddha"? **Follow-up Question 3**: (nl) Wie is de beroemde Koreaanse Go-speler met de bijnaam "Stone Buddha"? **Follow-up Question 4**: (fr) Qui est le célèbre joueur de go coréen surnommé "Stone Buddha" ? **Corrected Answer**: Lee Changho	**Follow-up Answer 1**: Lee Chang-ho **Follow-up Answer 2**: Lee Chang-ho **Follow-up Answer 3**: Cho Hun-hyun **Follow-up Answer 4**: Lee Chang-ho
ECMR3 Example	
Follow-up Question 1: Which Korean Go player is known as the "Stone Buddha"? **Follow-up Question 2**: Which Korean Go player is known by the nickname "Stone Buddha"? **Follow-up Question 3**: Who is the well-known Korean Go player known as the "Stone Buddha"? **Corrected Answer**: Lee Changho	**Follow-up Answer 1**: Choi Chul-han **Follow-up Answer 2**: Lee Chang-ho **Follow-up Answer 3**: Lee Changho
ECMR4 Example	
Follow-up Question 1: Known: [Cho Chihun in Korean, is among ...] May I ask: [Who is the famous ...] **Follow-up Question 2**: Known: [He is known as "Dol-bucheo" ...] May I ask: [Who is the famous ...] **Follow-up Question 3**: Known: [nicknamed "the God of Go" for ...] May I ask: [Who is the famous ...] **Corrected Answer**: Cho Hun-hyun	**Follow-up Answer 1**: Cho Chihun **Follow-up Answer 2**: Cho Hun-hyun **Follow-up Answer 3**: Cho Hun-hyun

ECMR3: Multi-Path QMR3. ECMR3 implements a multi-path version of QMR3. QMR3 changes both the words and structure of the original question without changing its semantic meanings. ECMR3 aims to introduce two new execution paths by: 1) synonymous substitutions of words under the condition that the structure of the original question is left as unaltered as possible, and 2) synonymous substitutions of the structure under the condition that the words of the original question are left as unaltered as possible. These three follow-up questions may induce the model to respond with different execution paths corresponding to different levels of question comprehension.

ECMR4: Multi-Path QMR4. ECMR4 implements a multi-path version of QMR4. ECMR4 adds new evidence to introduce multiple execution paths. New evidence is obtained by replacing the entities or words used for searching. Questions constructed from three different kinds of evidence have the potential to induce the model to respond with different execution paths corresponding to different knowledge recall processes.

ECMR5: Multi-Path CMR3. ECMR5 implements a multi-path version of CMR3. CMRs adjust multiple model capabilities, such as question comprehension, knowledge recall, and knowledge reasoning, to increase the possibility of activating different execution paths. Building upon the idea of CMRs, ECMR5 constructs four follow-up questions that can introduce significantly different execution paths than the original question, while retaining the meaning of the original question. ECMR5 combines ECMR2, ECMR3, ECMR4, and QMR1 to transform the original question. Specifically, ECMR5 first constructs an evidence pool, where the evidence is retrieved with ECMR4. Secondly, ECMR5 builds a question pool, which contains the original question and the three follow-up questions obtained by ECMR3. Thirdly, ECMR5 randomly selects four pieces of evidence from the evidence pool and combines them with each of the four questions in the question pool, respectively. Fourthly, ECMR5 adds a CoT prompt (similar to QMR1) to the above four combined questions and uses a language pool to translate these questions to obtain the final follow-up questions. The language pool is the same as that in ECMR2, which contains four languages: German, Dutch, Spanish, and French.

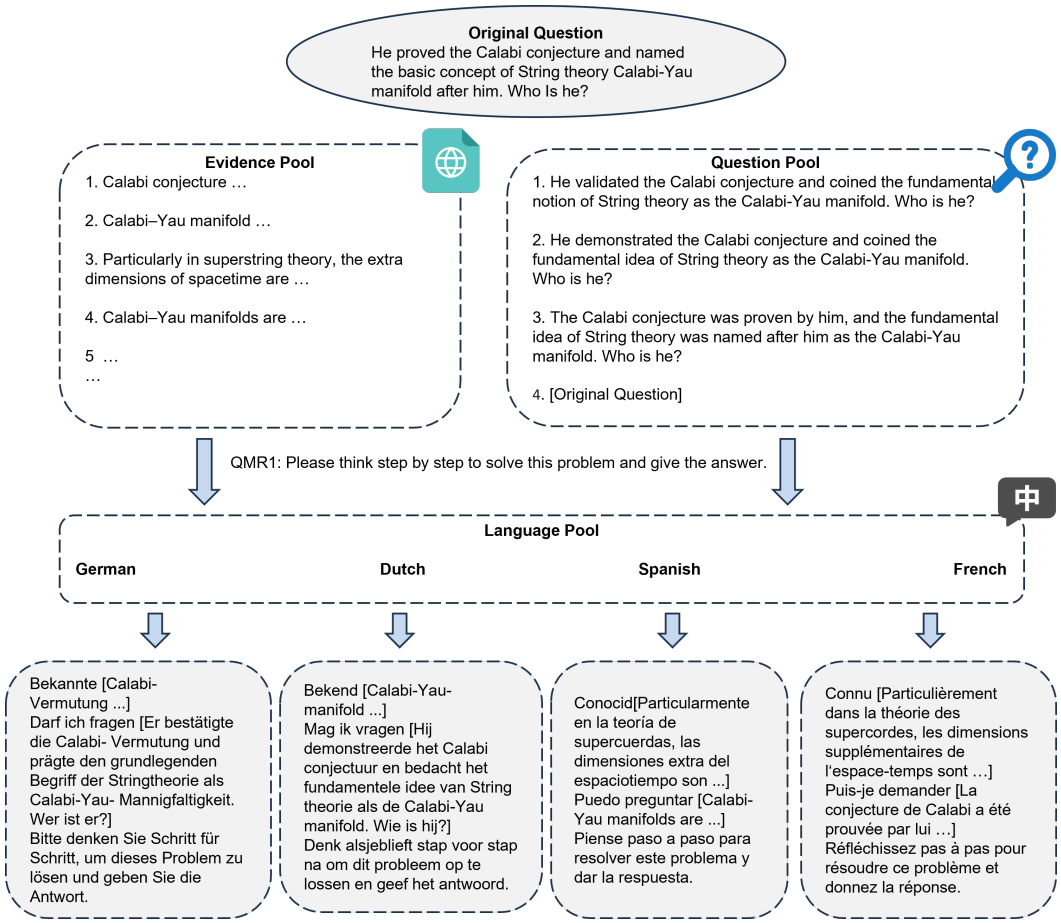


Fig. 4. An overview of the construction of ECMR5.

Following the same method in the consistency assessment process for answers, we process the outputs of models given the follow-up questions generated by ECMRs to obtain the follow-up answers. Particularly, we translate multilingual answers into English. For programming questions, we give the generated code the same input and run the code to obtain the outputs as the follow-up answers. We then have a vote on the follow-up answers, and the answer with the most votes is selected as the corrected answer. For programming questions, the voted answer is the test output of the generated code. Therefore, when multiple codes can generate the same test output with the most votes, we randomly select one from these code answers as the corrected answer. Besides, when there is a tie vote, we randomly select one from all the candidate answers with the most votes as the corrected answer.

4 Experimental Setup

In this section, we introduce our experimental setup to evaluate the effectiveness of our proposed framework, DrHall.

4.1 Research Questions

Our evaluation answers the following research questions (RQs):

RQ1: What is the quality of the self-built dataset? This RQ aims to evaluate the quality of the self-built dataset FactHalluQA to ensure its validity in subsequent experiments. The questions and reference answers in FactHalluQA are written by the authors. To ensure the validity of the dataset, we conducted several verifications.

RQ2: How effective are the proposed base and composite MRs for detecting factual hallucinations? This RQ aims to investigate the ability of DrHall to detect factual hallucinations. We consider both natural language and programming questions.

RQ3: How can the methodology be extended to correct factual hallucinations? This RQ aims to investigate the application of DrHall to correct factual hallucinations.

4.2 Test Subject

Our primary test subject is GPT-3.5-turbo. OpenAI's GPT-3.5-turbo is a black-box LLM and represents a noteworthy milestone in the progress of transformer-based language models [OpenAI Inc. 2023]. Due to its profound impacts, we choose to study GPT-3.5-turbo to ensure the significance of our work. Furthermore, we consider Mistral-7B and Llama-3.1-8B [Grattafiori et al. 2024; Jiang et al. 2023], which are representative open-source models. We also select GPT-4o, which is the most advanced LLM. As such, we can validate DrHall's generalization capability in dealing with the factual hallucinations of LLMs.

4.3 Datasets

We use three datasets covering questions in both natural language and programming language.

FactHalluQA. We use a self-constructed dataset FactHalluQA for natural language QA tasks. FactHalluQA consists of more than eight hundred questions in English, covering diverse subjects such as physics, chemistry, biology, geography, history, and literature. It is possible that LLMs' training data have included mainstream QA datasets, leading to the issue of benchmark leakage. Therefore, using the self-built dataset can address the benchmark leakage issue and characterize more realistic application scenarios. The construction process of FactHalluQA is described as follows:

(1) **Entity Filtering.** The diversity of the questions is crucial to ensure the generalizability of our results. Entities for each subject are selected mainly based on the 2019 National Unified Examination Outline for Higher Education Admissions [Ministry of Education of China 2019]. This ensures the diversity of the problems and makes them closer to the practical application scenarios of LLMs.
(2) **Fact Extraction.** To ensure the quality of the dataset, the questions and answers in the dataset are based on facts from Wikipedia. We search Wikipedia using the entities obtained in Step 1 and then extract factual statements from the corresponding Wikipedia pages.
(3) **Question and Answer Construction.** We transform the factual statements obtained in Step 2 into the form of questions and corresponding reference answers. During the construction process, we ensure that the reference answers to the obtained questions are correct and unambiguous.

Refactory. Refactory is a program repair dataset containing 2,442 correct programs and 1,783 incorrect programs on five programming tasks [Hu et al. 2019]. These data were collected from student assignments in a Python programming course at the National University of Singapore. In our experiments, we randomly select 30 correct programs and 30 incorrect programs for each programming task, constituting 300 program repair questions.

Table 4. The precision (P), recall (R), and F1 scores (F1) for Logprobs-Based with different thresholds. The test model is GPT-3.5-turbo.

(a) WH Question

Threshold	P	R	F1
-0.4	**0.871**	0.577	0.694
-0.3	0.831	0.632	**0.718**
-0.2	0.667	0.684	0.675
-0.1	0.490	**0.949**	0.646

(b) General or Choice Question

Threshold	P	R	F1
-0.18	**0.265**	0.455	0.335
-0.16	0.230	0.606	0.333
-0.14	0.260	0.848	0.399
-0.12	0.261	**0.955**	**0.410**

LeetCode. LeetCode is an online coding platform that includes over 2,000 questions in diverse categories, such as algorithms, databases, shells, and multithreading. In our experiments, we randomly select 200 questions labeled as "Hard" in the algorithm category and add the requirement of "generate in C++" or "generate in Python" to construct a total of 400 code generation questions.

4.4 Experimental Configurations

For GPT-3.5-turbo and GPT-4o, we use the OpenAI API to access them. For Mistral-7B, we select Mistral-7B-Instruct-v0.2 for the experiment. For Llama-3.1-8B, we select Llama-3.1-8B-Instruct for the experiment. We use the recommended hyper-parameters when processing QA tasks [OpenAI Inc. 2023]. Specifically, we set the max tokens to 100, the temperature to 0, the top-p to 1, the frequency penalty to 0, the presence penalty to 0, and the number of incidental history messages to 0. Among them, the temperature determines the randomness of the output, and setting it to 0 when processing the QA task ensures that the model gives more accurate answers. Since questions in our QA task are independent and do not involve a dialog, we set the number of incidental history messages to 0 in order to avoid the interference of history messages on the results.

4.5 Evaluation Metrics

We use the following metrics to assess the effectiveness of factual hallucination detection:

- **Precision.** Precision is the percentage of actual hallucinations identified by a detection method out of all the hallucinations it reports. It measures the extent to which a detection method can correctly report hallucinations. The higher the precision is, the more accurately the detection method can report hallucinations.
- **Recall.** Recall is the percentage of actual hallucinations identified by a detection method out of all actual hallucinations. It measures the extent to which a detection method can exhaustively identify hallucinations. The higher the recall is, the more exhaustively the detection method can identify hallucinations.
- **F1 Score.** The F1 score is the reconciled mean of precision and recall and is a relatively comprehensive evaluation metric. It takes into account both the precision and recall of a detection method.

For evaluating the performance of factual hallucination correction, we define the hallucination correction rate, which computes the percentage of successfully corrected answers over all hallucinated answers.

4.6 Baselines

In the context of detecting factual hallucinations, we compare DrHall with the following state-of-the-art baselines:

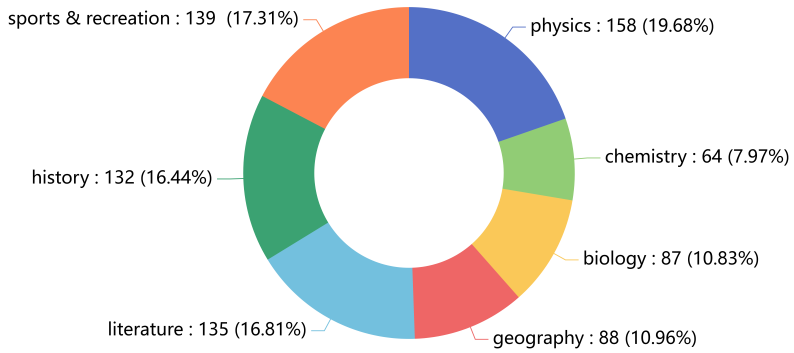


Fig. 5. The specific number of questions in our constructed dataset FactHalluQA for each subject.

Table 5. The hallucination detection results when dealing with natural language questions via our method. The test model is GPT-3.5-turbo. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	AMR2	CMR1	CMR2	CMR3
P	0.764	0.722	0.763	0.823	0.795	0.796	0.742	0.816	**0.832**
R	0.657	0.678	0.720	0.727	0.687	0.677	0.747	0.830	**0.840**
F1	0.706	0.700	0.741	0.772	0.737	0.732	0.744	0.823	**0.836**

QAAskeR. QAAskeR is a method for testing QA software based on recursive questioning [Chen et al. 2021]. It designs three metamorphic testing methods (QAAskeR MR1, QAAskeR MR2, and QAAskeR MR3) to check the consistency between the pairs of input questions and output answers related to the same pseudo-fact, where the input questions are of different types (e.g., general and WH questions), or where different objects are asked in the pseudo-fact. WH questions usually start with a word beginning with "wh-" but also include "how". The counterpart of this question type is the general or choice question, which is defined as a question that offers multiple options as answers. We follow the setup of the original paper to configure QAAskeR in our experiments. Note that QAAskeR only works for natural language questions.

SelfCheckGPT. SelfCheckGPT (SCG) is a strong baseline for detecting hallucinations of LLMs [Manakul et al. 2023]. It is based on the principle that if an LLM understands a concept, the sampled responses will be consistent and factually accurate. SCG uses four techniques: SCG-NLI, SCG-MQAG, SCG-BERT, and SCG-NGRAM. Each technique evaluates the consistency of the LLM's responses in a different way.

ChatProtect. ChatProtect is a prompt-based framework designed to effectively detect and mitigate self-contradiction [Mündler et al. 2023]. Its detector achieves high-precision detection of LLM hallucinations. We take the trigger module of the framework and follow the setup of the original paper to configure ChatProtect in our experiments.

Logprobs-Based. The LLM's prediction head outputs the probability of a predicted token. Using this probability as a measure of the LLM's confidence scores is a common gray-box method [Yoshikawa and Okazaki 2023]. When calling GPT-3.5-turbo's API, we can make the model output

Table 6. The hallucination detection results when dealing with natural language questions via different detection methods. The test model is GPT-3.5-turbo. QA-MR1, QA-MR2, and QA-MR3 denote different variants of QAAskeR. SCG-NLI, SCG-MQAG, SCG-BERT, and SCG-NGRAM denote different variants of SCG. CP denotes Chat-Protect. LB denotes Logprobs-Based. We report the precision (P), recall (R), and F1 scores (F1) for different methods.

	QA-MR1	QA-MR2	QA-MR3	SCG-NLI	SCG-MQAG	SCG-BERT	SCG-NGRAM	CP	LB	CMR3 (Ours)
P	0.609	0.623	0.336	0.585	0.444	0.470	0.486	0.716	0.648	**0.832**
R	0.502	0.425	0.636	0.725	0.651	0.700	0.633	0.597	0.735	**0.840**
F1	0.550	0.505	0.440	0.648	0.528	0.562	0.550	0.651	0.689	**0.836**

Table 7. The hallucination detection results when dealing with program repair questions via our method. The test model is GPT-3.5-turbo. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.800	**0.902**	0.864	0.874	0.785	0.795	0.870	0.878
R	0.641	0.702	0.634	0.740	**0.779**	0.679	0.718	0.771
F1	0.712	0.790	0.731	0.802	0.782	0.732	0.787	**0.821**

Table 8. The hallucination detection results when dealing with code generation questions via our method. The test model is GPT-3.5-turbo. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.882	0.895	0.898	0.932	0.870	0.908	0.891	**0.942**
R	0.831	**0.904**	0.872	0.885	0.886	0.879	0.872	0.879
F1	0.856	0.899	0.885	0.908	0.878	0.893	0.881	**0.910**

the log probability of each token. In our experiments, we compute the average log probability of only the answer tokens as the model's confidence scores. We set a threshold for the model's confidence scores to enable hallucination detection. The threshold is selected to maximize the F1 score. Table 4 shows the experiments for threshold selection. Based on our experimental results, we set the threshold to -0.3 for WH questions and -0.1 for general or multiple-choice questions. Therefore, Logprobs-Based will label answers with a confidence score less than the threshold as hallucinations.

In the context of correcting factual hallucinations, we select Self-Consistency (SC) [Wang et al. 2022] and Language Agent Tree Search (LATS) [Zhou et al. 2023a] as baselines. For SC, we follow the original paper and sample four CoT trajectories using a few-sample prompt at a decoding temperature of 0.7. For LATS, we adopt the experimental setup described in its original paper.

5 Results

5.1 RQ1

Most questions in our constructed dataset FactHalluQA are one sentence long with a median length of 15 words. Each sample includes a question, a reference answer, supporting evidence, and the source of the evidence. Figure 5 illustrates the statistics of the dataset. In summary, our FactHalluQA consists of 803 questions from seven subjects, including 158, 64, 87, 88, 135, 132, and 139 questions for physics, chemistry, biology, geography, literature, history, and sports & recreation, respectively.

Table 9. The hallucination detection results when dealing with natural language questions via our method. The test model is Mistral-7B-Instruct-v0.2. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	AMR2	CMR1	CMR2	CMR3
P	0.846	0.786	0.779	0.831	0.836	0.792	0.855	**0.870**	0.861
R	0.636	**0.835**	0.696	0.766	0.640	0.557	0.645	0.826	**0.835**
F1	0.726	0.810	0.735	0.797	0.725	0.654	0.735	**0.848**	**0.848**

Table 10. The hallucination detection results when dealing with program repair questions via our method. The test model is Mistral-7B-Instruct-v0.2. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.797	0.757	0.781	0.827	0.818	0.791	**0.881**	0.863
R	0.538	0.852	0.722	0.745	0.698	0.769	0.840	**0.863**
F1	0.642	0.802	0.750	0.784	0.753	0.780	0.860	**0.863**

Table 11. The hallucination detection results when dealing with code generation questions via our method. The test model is Mistral-7B-Instruct-v0.2. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.914	0.901	0.879	**0.946**	0.881	0.865	0.925	**0.946**
R	0.794	0.850	0.784	0.813	0.832	**0.841**	0.804	0.822
F1	0.850	0.875	0.808	0.874	0.856	0.853	0.860	**0.880**

The questions and reference answers in FactHalluQA were written by the authors. To ensure the dataset's quality, two external researchers, who were selected from university students and have English proficiency, knowledge retrieval tool proficiency, and sound judgment, were involved in a dataset validation process as described below:

(1) We provided the researchers with 250 questions and reference answers randomly sampled from FactHalluQA, along with supporting evidence for each question. Researchers are required to validate the data in terms of question syntax, ambiguity, and answer disagreement. The researcher must provide an explanation for any disagreement. They disagreed on 0.8% of all questions.
(2) Researchers were required to answer 250 questions randomly sampled from FactHalluQA. They may obtain the answers through any means, including searching the Internet. After comparing the participants' answers with the reference answers, we found a 2.8% disagreement.

These results suggest that external researchers disagree on few data in FactHalluQA. Upon analysis, we find that the disagreement primarily arises from the researchers' answers being either too general or too specific, resulting in an incomplete match with the reference answer.

Answer to RQ1. Our constructed natural language QA dataset FactHalluQA is of high quality. In external validation, researchers disagreed on few data in FactHalluQA. This ensures the validity of FactHalluQA in subsequent experiments.

Table 12. The hallucination detection results when dealing with natural language questions via our method. The test model is GPT-4o. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	AMR2	CMR1	CMR2	CMR3
P	0.720	0.760	0.726	0.719	**0.763**	0.741	0.724	0.734	0.724
R	0.704	0.681	0.667	**0.778**	0.691	0.685	0.719	0.756	**0.778**
F1	0.712	0.719	0.695	0.747	0.725	0.712	0.721	0.745	**0.750**

Table 13. The hallucination detection results when dealing with program repair questions via our method. The test model is GPT-4o. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.870	0.880	0.808	0.870	0.885	**0.913**	0.846	**0.913**
R	0.833	**0.917**	0.875	0.833	0.793	0.875	**0.917**	0.875
F1	0.851	**0.898**	0.840	0.851	0.836	0.894	0.880	0.894

Table 14. The hallucination detection results when dealing with code generation questions via our method. The test model is GPT-4o. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.900	0.756	0.750	0.790	**0.946**	0.738	0.790	0.723
R	0.729	0.837	0.729	0.811	0.714	0.838	0.811	**0.919**
F1	0.806	0.795	0.739	0.800	**0.814**	0.785	0.800	0.810

5.2 RQ2

In this section, we validate the effectiveness of DrHall in detecting factual hallucinations for different QA tasks. Among the baseline methods, QAAskeR MR1 and QAAskeR MR2 are only applicable to 546 WH questions in FactHalluQA, and QAAskeR MR3 is only applicable to 257 general or choice questions in FactHalluQA. All baseline methods and our AMR2 cannot be applied to programming QA tasks in Refactory and LeetCode.

Table 5 reports the performance of our MRs in detecting factual hallucinations for natural language QA tasks. The best base MR is QMR4, which achieves an F1 score of 0.772. The best composite MR is CMR3, whose F1 score can even reach 0.836. Table 6 compares our best MR with the state-of-the-art baselines. We can see that all our MRs can consistently outperform all the baselines, and our best MR can surpass the state-of-the-art baselines by a large margin, in terms of the obtained F1 score. Table 7 and Table 8 confirm that our MRs are also applicable to programming QA tasks and can obtain outstanding performance. Specifically, we can achieve an F1 score of 0.821 and 0.910 on Refactory and LeetCode, respectively.

When comparing different MRs, we can find that composite MRs usually exhibit stronger hallucination detection performance than component MRs. Besides, when more base MRs are combined, the resultant composite MRs can manifest better hallucination detection performance. Therefore, we think that the changes in the model's question comprehension, knowledge recall, and knowledge reasoning abilities incurred by MRs are superimposable, which makes it more likely to trigger different execution paths of the model with more combined MRs.

Table 15. The hallucination detection results when dealing with natural language questions via our method. The test model is Llama-3.1-8B-Instruct. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	AMR2	CMR1	CMR2	CMR3
P	0.695	0.717	0.656	0.754	0.744	0.705	0.736	0.769	**0.789**
R	0.819	0.759	**0.881**	0.848	0.648	0.770	0.752	0.826	0.840
F1	0.752	0.737	0.752	0.798	0.693	0.736	0.744	0.796	**0.814**

Table 16. The hallucination detection results when dealing with program repair questions via our method. The test model is Llama-3.1-8B-Instruct. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.652	0.779	0.692	0.717	**0.902**	0.709	0.790	0.756
R	0.811	0.756	0.866	0.836	0.671	0.933	0.833	**0.944**
F1	0.723	0.767	0.769	0.772	0.769	0.806	0.811	**0.840**

Table 17. The hallucination detection results when dealing with code generation questions via our method. The test model is Llama-3.1-8B-Instruct. We report the precision (P), recall (R), and F1 scores (F1) for different MRs of our method.

	QMR1	QMR2	QMR3	QMR4	AMR1	CMR1	CMR2	CMR3
P	0.829	0.836	0.863	0.875	**0.918**	0.861	0.888	0.893
R	0.816	0.882	0.902	0.892	0.853	**0.953**	0.897	0.933
F1	0.822	0.858	0.882	0.884	0.884	0.905	0.893	**0.912**

To validate the generalizability of DrHall in detecting factual hallucinations of LLMs, we also conduct experiments with different LLMs across different datasets in Tables 9–17. Specifically, we cover Mistral-7B, Llama-3.1-8B, and GPT-4o. The experimental results confirm that our method can still effectively detect factual hallucinations for different LLMs.

Answer to RQ2. The proposed MRs achieve outstanding factual hallucination detection performance for both natural language and programming QA tasks across different LLMs. Among base MRs, QMR4 often performs the best. Besides, when more base MRs are combined, the resultant composite MRs can manifest better hallucination detection performance. Therefore, among all MRs, CMR3 usually performs the best.

5.3 RQ3

We randomly select questions whose original answers are hallucinations from our datasets to test the factual hallucination correction performance of our method. The selected questions include 100 WH questions, 50 general or choice questions, 50 program repair questions, and 50 code generation questions. The experimental results on GPT-3.5-turbo are shown in Table 18.

We can see that our method can outperform the state-of-the-art baselines by a significant margin. Besides, ECMR5 has the best overall performance for hallucination correction, which achieves a 60% hallucination correction rate for natural language QA tasks. We think that the probability of generating the same hallucination with more distinct execution paths of models is lower, and sampling from different execution paths of models can increase the probability of obtaining the

Table 18. The hallucination correction results when dealing with different QA tasks via different methods. We report the hallucination correction rates for baselines (SC and LATS) and different ECMRs of our method.

Question	SC	LATS	ECMR1	ECMR2	ECMR3	ECMR4	ECMR5
Natural Language	43%	56%	37%	43%	49%	53%	**60%**
Program Repair	34%	42%	38%	48%	56%	42%	**62%**
Code Generation	24%	39%	34%	**42%**	40%	38%	38%

correct answer. Since ECMR5 can introduce the most different execution paths of models, it can obtain the best overall performance for hallucination correction.

Answer to RQ3. Our experimental results show that we can extend DrHall to correct factual hallucinations of LLMs with multi-path voting. Generally, our designed ECMR5 based on composite MRs has the best hallucination correction performance, achieving a 60% hallucination correction rate for natural language QA tasks.

5.4 Threats to Validity

The **internal threats** to validity arise from the inherent randomness in the responses generated by large language models. The stochastic nature of LLMs can introduce variability and biases into the experimental outcomes, potentially skewing the results. In order to mitigate this threat, we repeated our experiments three times and reported the average results to reduce the impact of randomness and increase the reliability of our findings.

The **external threats** to validity arise from the limited number of test subjects we have chosen. More test subjects could potentially enhance the generalizability of our findings. Nevertheless, we have selected four representative LLMs in our experiments, which include both proprietary and open-source models and are also widely used in the literature. We plan to test our method with more LLMs in future work to further validate its generalizability.

6 Conclusion

We introduce DrHall, a framework for detecting and reducing the factual hallucinations of black-box LLMs with metamorphic testing. We design metamorphic relations based on the capabilities used by LLMs when dealing with QA tasks. To detect factual hallucinations of LLMs, we devise six base metamorphic relations and three composite metamorphic relations. To correct factual hallucinations of LLMs, we devise five ECMRs. To avoid benchmark leakage, we construct a new natural language QA dataset FactHalluQA. Besides, we test our framework with programming QA datasets to evaluate its performance against code hallucinations. Our results show that DrHall can significantly outperform the state-of-the-art baselines in terms of hallucination detection and correction.

7 Data Availability

Our artifacts are available at https://github.com/YuhanggCao/DrHall.

Acknowledgments

We would like to extend our sincere gratitude to all the anonymous reviewers for their thoughtful and constructive feedback. This work was supported by the National Natural Science Foundation of China (Grant Nos. 62206318 and 62032025).

References

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565* (2016).

Apple Inc. 2023. Apple Siri. https://www.apple.com/siri/.

Baidu Inc. 2023. Baidu Xiaodu. https://dumall.baidu.com/global/.

Iñigo López-Riobóo Botana, Dana Gallent-Iglesias, and Sonia González-Vázquez. 2023. QUA4I: Question answering for the Industry 4.0 domain. An application of intelligent virtual assistants. In *Annual Conference of the Spanish Society for Natural Language Processing*. https://api.semanticscholar.org/CorpusID:264491948

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

Canyu Chen and Kai Shu. 2024. Can LLM-generated misinformation be detected? *International Conference on Learning Representations* (2024). https://openreview.net/forum?id=ccxD4mtkTU

Jifan Chen, Grace Kim, Aniruddh Sriram, Greg Durrett, and Eunsol Choi. 2024. Complex claim verification with evidence retrieved in the wild. *North American Chapter of the Association for Computational Linguistics* (2024), 3569–3587. doi:10.18653/v1/2024.naacl-long.196

Songqiang Chen, Shuo Jin, and Xiaoyuan Xie. 2021. Testing your question answering software via asking recursively. In *IEEE/ACM International Conference on Automated Software Engineering*. 104–116. doi:10.1109/ASE51524.2021.9678670

Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. 2020. Metamorphic testing: a new approach for generating next test cases. *arXiv preprint arXiv:2002.12543* (2020).

Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *Comput. Surveys* 51, 1 (2018), 1–27. doi:10.1145/3143561

I Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan, Kehua Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. 2023. FacTool: Factuality detection in generative AI–a tool augmented framework for multi-task and multi-domain scenarios. *arXiv preprint arXiv:2307.13528* (2023).

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044* (2019).

Yang Deng, Weibin Wu, Jianping Zhang, and Zibin Zheng. 2023. Blurred-Dilated method for adversarial attacks. In *Advances in Neural Information Processing Systems*, Vol. 36. 58613–58624.

Ashwin Devaraj, William Sheffield, Byron C Wallace, and Junyi Jessy Li. 2022. Evaluating factuality in text simplification. In *The 60st Annual Meeting of the Association for Computational Linguistics*. 7331. doi:10.18653/v1/2022.acl-long.506

Sashank Gondala, Lyan Verwimp, Ernest Pusateri, Manos Tsagkias, and Christophe Van Gysel. 2021. Error-driven pruning of language models for virtual assistants. In *IEEE International Conference on Acoustics, Speech and Signal Processing*. 7413–7417.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. 2022. A survey on automated fact-checking. *Transactions of the Association for Computational Linguistics* 10 (2022), 178–206. doi:10.1162/tacl_a_00454

Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. 2021. Unsolved problems in ML safety. *arXiv preprint arXiv:2109.13916* (2021).

Lynette Hirschman and Robert Gaizauskas. 2001. Natural language question answering: The view from here. *natural language engineering* 7, 4 (2001), 275–300.

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-factoring based program repair applied to programming assignments. In *IEEE/ACM International Conference on Automated Software Engineering*. 388–398. doi:10.1109/ASE.2019.00044

Kung-Hsiang Huang, Hou Pong Chan, and Heng Ji. 2023. Zero-shot faithful factual error correction. In *The 61st Annual Meeting of the Association for Computational Linguistics*. 5660–5676. doi:10.18653/v1/2023.acl-long.311

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.* 43, 2 (2025). doi:10.1145/3703155

Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R Lyu. 2024. Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials. *The ACM on Software Engineering* 1, FSE (2024), 2515–2537. doi:10.1145/3660818

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221* (2022).

Byron Kaye. 2023. Australian mayor readies world's first defamation lawsuit over ChatGPT content. https://www.reuters.com/technology/australian-mayor-readies-worlds-first-defamation-lawsuit-over-chatgpt-content-2023-04-05/.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems* 35 (2022), 22199–22213.

Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2014. How effectively does metamorphic testing alleviate the oracle problem? *IEEE Transactions on Software Engineering* 40, 1 (2014), 4–22. doi:10.1109/TSE.2013.46

Jiacheng Liu, Alisa Liu, Ximing Lu, Sean Welleck, Peter West, Ronan Le Bras, Yejin Choi, and Hannaneh Hajishirzi. 2022. Generated knowledge prompting for commonsense reasoning. *The 60th Annual Meeting of the Association for Computational Linguistics* (2022), 3154–3169. doi:10.18653/v1/2022.acl-long.225

Junyu Luo, Cao Xiao, and Fenglong Ma. 2023. Zero-resource hallucination prevention for large language models. *arXiv preprint arXiv:2309.02654* (2023).

Quang-Hung Luu, Huai Liu, Tsong Yueh Chen, and Hai L Vu. 2022. A sequential metamorphic testing framework for understanding automated driving systems. *arXiv preprint arXiv:2206.03075* (2022).

Potsawee Manakul, Adian Liusie, and Mark JF Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. *Conference on Empirical Methods in Natural Language Processing* (2023), 9004–9017. doi:10.18653/v1/2023.emnlp-main.557

Ning Miao, Yee Whye Teh, and Tom Rainforth. 2023. Selfcheck: Using LLMs to zero-shot check their own step-by-step reasoning. *arXiv preprint arXiv:2308.00436* (2023).

Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. *Empirical Methods in Natural Language Processing* (2023), 12076–12100. doi:10.18653/v1/2023.emnlp-main.741

Ministry of Education of China. 2019. China's national college entrance examination. https://gaokao.neea.edu.cn/html1/category/1509/6212-1.htm.

Niels Mündler, Jingxuan He, Slobodan Jenko, and Martin Vechev. 2023. Self-contradictory hallucinations of large language models: Evaluation, detection and mitigation. *arXiv preprint arXiv:2305.15852* (2023).

OpenAI Inc. 2023. Introducing ChatGPT. https://openai.com/blog/chatgpt.

Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov. 2024. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. *arXiv preprint arXiv:2410.02707* (2024).

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.

Andrés Piñeiro-Martín, Carmen García-Mateo, Laura Docío-Fernández, and María del Carmen López-Pérez. 2023. Ethical challenges in the development of virtual assistants powered by large language models. *Electronics* 12, 14 (2023), 3170.

Vikas Raunak, Matt Post, and Arul Menezes. 2022. Salted: A framework for salient long-tail translation error detection. *arXiv preprint arXiv:2205.09988* (2022).

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324* (2023).

Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R Johnston, et al. 2023. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548* (2023).

Qingchao Shen, Junjie Chen, Jie M Zhang, Haoyu Wang, Shuang Liu, and Menghan Tian. 2022. Natural test generation for precise testing of question answering software. In *IEEE/ACM International Conference on Automated Software Engineering*. 1–12. doi:10.1145/3551349.3556953

Derek Tam, Anisha Mascarenhas, Shiyue Zhang, Sarah Kwan, Mohit Bansal, and Colin Raffel. 2022. Evaluating the factual consistency of large language models through summarization. *arXiv preprint arXiv:2211.08412* (2022).

Yiming Tan, Dehai Min, Yu Li, Wenbo Li, Nan Hu, Yongrui Chen, and Guilin Qi. 2023. Evaluation of ChatGPT as a question answering system for answering complex questions. *arXiv preprint arXiv:2303.07992* (2023).

Yuchen Tian, Weixiang Yan, Qian Yang, Qian Chen, Wen Wang, Ziyang Luo, and Lei Ma. 2024. CodeHalu: Code hallucinations in LLMs driven by execution-based verification. *arXiv preprint arXiv:2405.00253* (2024).

Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. 2023. A stitch in time saves nine: Detecting and mitigating hallucinations of LLMs by validating low-confidence generation. *arXiv preprint arXiv:2307.03987* (2023).

Cunxiang Wang, Xiaoze Liu, Yuanhao Yue, Xiangru Tang, Tianhang Zhang, Cheng Jiayang, Yunzhi Yao, Wenyang Gao, Xuming Hu, Zehan Qi, et al. 2023. Survey on factuality in large language models: Knowledge, retrieval and domain-specificity. *arXiv preprint arXiv:2310.07521* (2023).

Shuai Wang and Zhendong Su. 2020. Metamorphic object insertion for testing object detection systems. In *IEEE/ACM International Conference on Automated Software Engineering*. 1053–1065.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

Yuxia Wang, Minghan Wang, Hasan Iqbal, Georgi Georgiev, Jiahui Geng, and Preslav Nakov. 2024. OpenFactCheck: A unified framework for factuality evaluation of LLMs. *Empirical Methods in Natural Language Processing: System Demonstrations* (2024), 219–229. doi:10.18653/v1/2024.emnlp-demo.23

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

Han Wu, Guanyan Ou, Weibin Wu, and Zibin Zheng. 2024. Improving transferable targeted adversarial attacks with model self-enhancement. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 24615–24624. doi:10.1109/CVPR52733.2024.02324

Weibin Wu, Jianping Zhang, Victor Junqiu Wei, Xixian Chen, Zibin Zheng, Irwin King, and Michael R Lyu. 2023. Practical and efficient model extraction of sentiment analysis APIs. In *International Conference on Software Engineering*. 524–536. doi:10.1109/ICSE48619.2023.00054

Caiming Xiong, Victor Zhong, and Richard Socher. 2016. Dynamic coattention networks for question answering. *arXiv preprint arXiv:1611.01604* (2016).

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. *International Conference on Learning Representations* (2024). https://openreview.net/forum?id=gjeQKFxFpZ

Hiyori Yoshikawa and Naoaki Okazaki. 2023. Selective-LAMA: Selective prediction for confidence-aware evaluation of language models. In *European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, 2017–2028. doi:10.18653/v1/2023.findings-eacl.150

Jianping Zhang, Yung-Chieh Huang, Weibin Wu, and Michael R Lyu. 2023. Towards semantics-and domain-aware adversarial attacks. In *International Joint Conference on Artificial Intelligence*. 536–544. doi:10.24963/ijcai.2023/60

Yiran Zhao, Jinghan Zhang, I Chern, Siyang Gao, Pengfei Liu, Junxian He, et al. 2023. Felm: Benchmarking factuality evaluation of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 44502–44523.

Shen Zheng, Jie Huang, and Kevin Chen-Chuan Chang. 2023. Why does ChatGPT fall short in answering questions faithfully? *arXiv preprint arXiv:2304.10513* (2023).

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023a. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406* (2023).

Kun Zhou, Yutao Zhu, Zhipeng Chen, Wentong Chen, Wayne Xin Zhao, Xu Chen, Yankai Lin, Ji-Rong Wen, and Jiawei Han. 2023b. Don't make your LLM an evaluation benchmark cheater. *arXiv preprint arXiv:2311.01964* (2023).

Received 2024-09-13; accepted 2025-01-14